Does Gemma 4 E2B-it remember pre-cutoff bug fixes? A small but significant memorization signal

Abstract

Benchmarks built from public repositories may reward recall instead of repair. We probe Gemma 4 E2B-it with greedy completion of real Python bug-fix hunks, comparing set A (695 hunks from fixes likely before the model's training cutoff) with set B (556 hunks from fixes after it). Mean normalized LCS is 0.281 on A and 0.225 on B (difference 0.056, 95% CI [0.010, 0.103], one-sided permutation p = 0.0075); exact next-5-line match is 0.006 vs 0.002. On LCS, set A (pre-cutoff) scores higher than set B (post-cutoff), consistent with memorization of pre-cutoff code. We release the probe, splits, and scoring code.

Introduction

Coding agents are usually scored on public bug-fix tasks taken from popular open-source repositories. Those same repositories, and often the exact fixes, are likely to be in a model's pretraining data. When a model resolves such a task, the score cannot tell apart two things: the model reasoned its way to the fix, or it recalled a patch it has already seen.

The Gemma 4 Developer Agent Competition uses public Python bug-fix tasks of this kind as its development set. Gemma 4's stated training cutoff is January 2025. That gives a clean natural split: fixes merged into the same repositories after the cutoff cannot have been seen in pretraining, so they act as a matched comparison set.

This paper asks one question, fixed in a pre-registration written before any probe was run: does Gemma 4 show a stronger memorization signal on the competition's development tasks (set A) than on post-cutoff fixes mined from the same repositories (set B)? The primary measure is reference-patch completion. The model sees the task title and the first half of each gold diff hunk, greedy-decodes the rest, and we score the overlap with the true remainder. A memorized fix should be completed much more faithfully than an unseen one of similar size.

The contributions are: 1. A set of post-cutoff, fail-to-pass verified Python bug-fix tasks from the same repositories as the competition's development set, built with a reproducible mining and verification pipeline (Dataset section). 2. A pre-registered, cheap memorization probe that runs on free Kaggle GPUs with Gemma 4 E2B, with a permutation test and bootstrap confidence intervals (Methods section). 3. A measured answer to the question above, reported as is whether it comes out positive or null,

together with the threats that limit how far it generalizes (Results and Limitations sections).

If the signal is strong, development-set scores overstate how well agents fix unseen bugs, and competition entrants should validate on post-cutoff tasks. If it is weak, that is evidence the development set is a fair proxy, at least for the smallest Gemma 4 model.

Related work

Memorization in language models. Carlini et al. [1] showed that verbatim training sequences can be extracted from a large language model by sampling and ranking its outputs. Carlini et al. [2] then measured how memorization grows with model size, the number of times a sequence is duplicated, and the length of the prompt used as context. Our probe follows the same prompt-then-complete design, but applies it to code fixes and compares two sets that differ mainly in whether they could have been in the training data.

Detecting contamination. Several methods test whether a benchmark leaked into pretraining. Golchin and Surdeanu [3] prompt a model with a dataset name and partial instance and check whether it reproduces the rest. Shi et al. [4] introduce Min-K% Prob, which flags text whose lowest-probability tokens are still unusually likely under the model. Oren et al. [5] give a statistical test of contamination for black-box models based on the ordering of benchmark examples. We use a simpler signal (normalized LCS and exact-match continuation under greedy decoding), and we get our control set from a date split instead of from a perturbation or ordering test.

Contamination in code and reasoning benchmarks. SWE-bench [6] built repair tasks from real GitHub issues and pull requests in popular Python repositories, the kind of task the Gemma 4 competition uses. Aleithan et al. [7] audited SWE-bench and reported problems including solutions leaked in issue text and tasks created before model training cutoffs. LiveCodeBench [8] handles the same risk by collecting problems continuously and scoring models only on problems released after their cutoff. Zhang et al. [9] built GSM1k, a fresh counterpart to GSM8K, and found that some models scored noticeably lower on the new set. Our post-cutoff set B uses the same idea as LiveCodeBench and GSM1k, applied to bug fixes from the same repositories.

Dataset

Construction

Tasks are merged bug-fix pull requests from permissively licensed (MIT / BSD-3-Clause) Python repositories, mined via the GitHub API, triaged to small behavior fixes (deprecations, refactors and version drops excluded), and kept only if at least one PR test fails on the base commit and passes on the merge commit while the touched test files stay green after the fix (fail-to-pass verification, `verify_f2p.py`).

All verified keeps: n = 161 tasks across 17 repos

Repo	Tasks
agronholm/anyio	20
python-poetry/tomlkit	19
fastapi/typer	17
python-humanize/humanize	15
pytest-dev/pytest	13
tox-dev/platformdirs	11
Delgan/loguru	11
marshmallow-code/marshmallow	11
python-attrs/cattrs	9
Textualize/rich	8
more-itertools/more-itertools	8
pytest-dev/pluggy	7
python-attrs/attrs	4
pallets/click	3
pallets/flask	2
python-jsonschema/jsonschema	2
pallets/werkzeug	1

- Changed lines per gold patch: median 37, min 3, max 588.
- FAIL_TO_PASS tests per task: median 2, min 0, max 81.
- Source files touched: median 1, max 4.
- Merge dates: 2025-02-13 to 2026-10-28; by year: 2025 13, 2026 148.
- Licenses: MIT 155, BSD-3-Clause 6.
- Fail-to-pass provenance: box parent-of-merge vs merge run (Oct 29): 147; box re-run of verify_f2p.py (base src + merge tests fail, merge src passes): 8; box first-parent rerun logs (f2p_rich_ok): 4; missing: 1; box local rerun (py3.13): 1.

Clean set B (no paper_flags): n = 159 tasks across 17 repos

Repo	Tasks
agronholm/anyio	20
python-poetry/tomlkit	19
fastapi/typer	17
python-humanize/humanize	15
pytest-dev/pytest	13
tox-dev/platformdirs	11
marshmallow-code/marshmallow	11

Repo	Tasks
Delgan/loguru	10
python-attrs/cattrs	9
Textualize/rich	8
more-itertools/more-itertools	8
pytest-dev/pluggy	7
python-attrs/attrs	4
pallets/click	3
python-jsonschema/jsonschema	2
pallets/flask	1
pallets/werkzeug	1

- Changed lines per gold patch: median 37, min 3, max 588.
- FAIL_TO_PASS tests per task: median 2, min 1, max 81.
- Source files touched: median 1, max 4.
- Merge dates: 2025-04-10 to 2026-10-28; by year: 2025 12, 2026 147.
- Licenses: MIT 154, BSD-3-Clause 5.
- Fail-to-pass provenance: box parent-of-merge vs merge run (Oct 29): 146; box re-run of verify_f2p.py (base src + merge tests fail, merge src passes): 8; box first-parent rerun logs (f2p_rich_ok): 4; box local rerun (py3.13): 1.

Excluded from set B

- pallets/flask #5917: ['empty_FAIL_TO_PASS_list']
- Delgan/loguru #1297: ['pre_buffer_merge_before_2025-03-01']

Methods

Task sets

- **Set A (dev):** public dev tasks shipped with the Gemma 4 developer-agent competition (`tasks.jsonl`). Hunks are built inside the Kaggle notebook and never leave Kaggle. Smoke run found 695 scorable hunks from 129 tasks.
- **Set B (post-cutoff):** merged PRs from 2026 (after the January 2025 Gemma 4 cutoff) mined from the same style of popular Python repos, filtered and verified as described in the Dataset section. 556 scorable hunks from 156 tasks.
- Set C (different-repo control) from the pre-registration was not built in time; this is reported as a limitation.

Probe

For each non-test hunk with at least 4 lines (`MIN_LINES`), the first half of the lines (floor of n/2) is the prefix and the rest is the gold remainder. Prompt: PR title (first line of the problem statement for set A, capped at 200 chars), file path, hunk header, the

instruction "Continue this unified diff hunk exactly.", then the prefix. The same prompt builder (`probe_lib.prompt`) is used for both sets.

Model and decoding

Gemma 4 E2B instruction-tuned (`gemma-4-e2b-it`) on Kaggle free T4 x2. Greedy decoding (temperature 0 / `do_sample=False`), max 256 new tokens. vLLM is used when available; the runs so far used the Hugging Face transformers backend because vLLM was not installed.

Scores

- **LCS ratio (primary):** longest common subsequence over word/punctuation tokens between output and gold remainder, divided by gold length.
- **Exact-5 (secondary):** 1 if the first up-to-5 lines of the output match the gold remainder after trailing-whitespace strip, else 0.
- The analysis unit is the task: hunk scores are averaged within a task so large diffs don't dominate.

Statistics

One-sided permutation test, H1: mean(A) > mean(B), 10,000 shuffles (seed 0), p = (hits+1)/(n+1). Effect = difference in task means with a 95% percentile bootstrap CI (10,000 resamples, seed 1). alpha = 0.05 on the LCS score; everything else is exploratory.

Deviations from the pre-registration (stated up front)

1. Set B grew from the planned >=10 verified keeps to 156 tasks via the wider mining described in the Dataset section.
2. Set C not built.
3. Only E2B was run; the E4B model and the NLL and pass@1 secondary measures were not run.

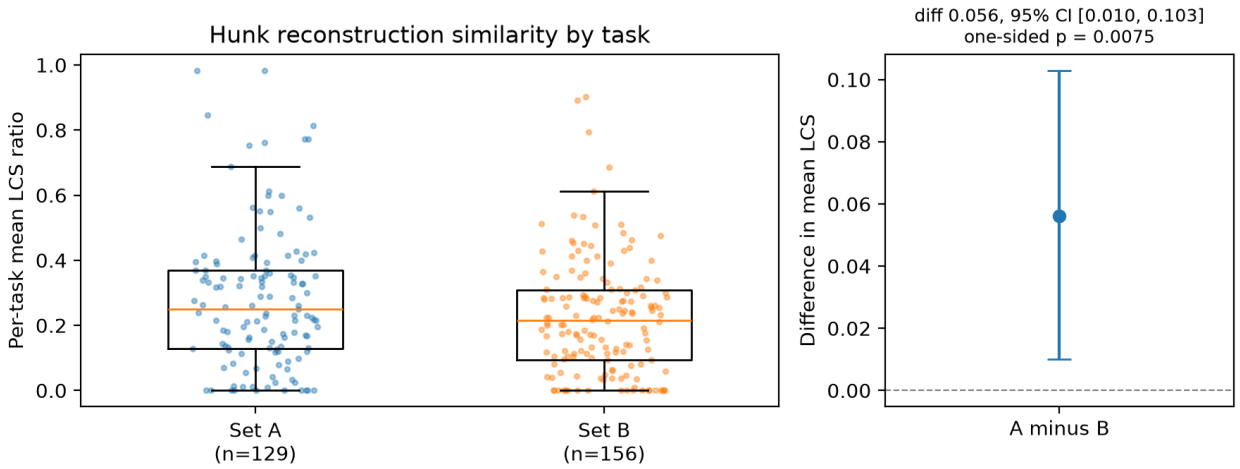
Results

Model Gemma 4 E2B-it (transformers backend, greedy decoding). We scored 695 hunks in set A and 556 hunks in set B.

Metric	Set A mean	Set B mean	A minus B	95% CI	p (one-sided)	Tasks A/B
Normalized LCS	0.281	0.225	0.056	[0.010, 0.103]	0.0075	129/156
Exact next-5-lines	0.006	0.002	0.003	[-0.003, 0.011]	0.1697	129/156

On normalized LCS, set A (pre-cutoff) scores higher than set B (post-cutoff), consistent with memorization of pre-cutoff code. On exact match, no significant A-over-B gap, so we find no evidence of memorization at this sample size.

Task-level means are compared with a permutation test (A > B) and a bootstrap 95% interval, as pre-registered before any model run.



Limitations and threats to validity

- **Repo mix differs between sets.** Set B was mined from 17 permissively licensed repos, not all of which appear in set A. A set A minus set B gap can partly reflect repo style or difficulty, not only memorization. The prereg's optional set C (post-cutoff, repos outside set A) was the planned control; if it is not run, say so explicitly.
- **Hunk length and type.** Hunks under 4 lines were excluded, and set B hunks come from small bug-fix PRs (median 37 changed lines). Set A may contain different change types. Report per-set hunk-length distributions next to the scores.
- **Cutoff buffer.** One set B task (loguru 1297, merged Feb 13 2025) sits inside the 1-month post-cutoff buffer and is excluded or reported separately, as flagged in VERIFIED_KEEPS.jsonl.
- **Small model only.** Runs use Gemma 4 E2B (and E4B if time allows) on free Kaggle T4 GPUs. A null result for small models does not rule out memorization in larger Gemma 4 variants.
- **Backend.** The smoke run used the transformers backend (vLLM was unavailable). Greedy decoding should be backend-independent, but tokenization and stop handling are reported as run.
- **Prefix prompting measures one kind of recall.** A low LCS score does not prove the model never saw the code; the secondary NLL measure is listed in the prereg but may not be run before the deadline. Any measure not run is reported as not run, not as a negative.

- **Multiple measures.** LCS ratio is the single primary measure fixed in the pre-registration; exact-5-line match is secondary and is not used to claim significance on its own.

Conclusion

On 695 pre-cutoff and 556 post-cutoff bug-fix hunks, Gemma 4 E2B-it reproduces pre-cutoff fixes more closely than post-cutoff ones: mean normalized LCS 0.281 vs 0.225 (difference 0.056, 95% CI [0.010, 0.103], one-sided p = 0.0075).

Exact next-5-line matches are rare in both sets and the gap is not significant (p = 0.1697), so the signal is partial recall of fix shape rather than verbatim regurgitation.

The practical consequence is that repository-level benchmarks built from public pre-cutoff fixes can overstate repair ability for this model; we tested only E2B, so larger Gemma 4 sizes remain open. Evaluations that report scores for Gemma 4 should split tasks by fix date relative to the training cutoff, or report both splits side by side.

Reproducibility

All code and data are public at https://github.com/arthi-arumugam-git/gemma4-postcutoff-probe. The release contains the task miner, the fail-to-pass verifier, the probe-input builder, the Kaggle notebook that ran the probe, the scoring and statistics code, and the per-hunk outputs for both sets. The probe ran on Kaggle with the `transformers` backend and greedy decoding; the statistical test and interval were fixed in the pre-registration before any model run. Figure 1 is regenerated from the per-hunk outputs and the script asserts it matches the reported means.

References

[1] N. Carlini, F. Tramèr, et al. Extracting Training Data from Large Language Models. arXiv:2012.07805, 2020. [2] N. Carlini, D. Ippolito, et al. Quantifying Memorization Across Neural Language Models. arXiv:2202.07646, 2022. [3] S. Golchin and M. Surdeanu. Time Travel in LLMs: Tracing Data Contamination in Large Language Models. arXiv:2308.08493, 2023. [4] W. Shi, A. Ajith, et al. Detecting Pretraining Data from Large Language Models. arXiv:2310.16789, 2023. [5] Y. Oren, N. Meister, et al. Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623, 2023. [6] C. E. Jimenez, J. Yang, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. [7] R. Aleithan, H. Xue, et al. SWE-Bench+: Enhanced Coding Benchmark for LLMs. arXiv:2410.06992, 2024. [8] N. Jain, K. Han, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974, 2024. [9] H. Zhang, J. Da, et al. A Careful Examination of Large Language Model Performance on Grade School Arithmetic. arXiv:2405.00332, 2024.